

**POSTS AND TELECOMMUNICATIONS INSTITUTE OF
TECHNOLOGY**

Faculty of Information Technology

Intelligent System Development

ASSIGNMENT 06

**Recurrent Neural Networks on Time Series:
From Scratch, in Keras and in PyTorch**

Instructor / Lecturer: Assoc. Prof. Dinh Que Tran, Ph.D.
Email: quetd@ptit.edu.vn, tdque@yahoo.com

Student Name: Nguyễn Văn Trường
Student ID: B23DCCE095
Semester: I.2025 – 2026
Class: E23CNPM02
Submission: Individual Project

Hanoi, 2026

Contents

1	Overview	2
2	Theoretical Background of Recurrent Networks	3
2.1	Time-Series Data and the Forecasting Problem	3
2.2	Mathematical Operators	3
2.3	The Recurrent Cell	4
2.4	Training a Recurrent Network	4
2.5	The Vanishing and Exploding Gradient Problem	5
2.6	Gated Cells: LSTM and GRU	5
2.7	Evaluating a Forecast: Baselines and Metrics	6
2.8	The Same Layer in Three Implementations	6
3	Data: Sources, Features and Splits	7
3.1	EC: Order Flow of an Online Retailer (Online Retail II)	7
3.2	ST: Daily Returns of S&P 500 Stocks	7
3.3	Splitting, Scaling and Evaluation Protocol	8
4	Model Architecture and Parameters	8
4.1	Architecture	8
4.2	Hyperparameters and Configuration	9
4.3	Trainable Parameters	9
4.4	Initialisation and Naming	10
5	Implementation Details	10
5.1	NumPy Implementation from Scratch	10
5.2	Keras Implementation	10
5.3	PyTorch Implementation	11
5.4	Verification That the Three Implementations Agree	11
6	Experiment 1: Forecasting Order Volume (EC)	11
7	Experiment 2: Forecasting Stock Returns (ST)	13
8	Comparison of Implementations and Datasets	14
9	Conclusions and Limitations	15

1 Overview

This report studies the recurrent neural network (RNN) on two time-related datasets and implements the same model three times: **from scratch** in NumPy (SC, hand-written forward pass, back-propagation through time and Adam), in **Keras** (TF) and in **PyTorch** (PT). The datasets are **EC**, a service process of an e-commerce company (Online Retail II: orders and cancellations of 750 products over 604 open days, 432,000 windows) and **ST**, stock prices (S&P 500: 350 stocks over 1,258 trading days, 429,800 windows). Both tasks are many-to-one regression: a window of past days predicts the next day. All implementations share the data, the split, the architecture (one vanilla RNN layer with 32 units and a linear head), the starting weights, the optimizer and the early-stopping rule, so the only thing that changes is the abstraction level. LSTM and GRU are added as extras in Keras and PyTorch. Run IDs have the form `<dataset>-<implementation>-<model>`, for example ST-TF-GRU.

Table 1: All 14 runs, test period. RMSE in units sold (EC, top block) and in log-return (ST, bottom block); R^2 is out-of-sample against the train-mean predictor; s/epoch = median time per epoch on CPU.

Run	test RMSE	R^2	params	s/epoch
EC-SC-RNN	62.51	0.439	1,409	3.9
EC-TF-RNN	63.01	0.438	1,409	6.2
EC-PT-RNN	62.84	0.439	1,409	11.2
EC-TF-LSTM	62.44	0.446	5,537	14.6
EC-PT-LSTM	62.03	0.447	5,665	22.0
EC-TF-GRU	62.41	0.449	4,257	16.4
EC-PT-GRU	62.24	0.448	4,257	28.4
ST-SC-RNN	0.0159	-0.017	1,217	4.2
ST-TF-RNN	0.0159	-0.017	1,217	6.5
ST-PT-RNN	0.0158	-0.016	1,217	11.8
ST-TF-LSTM	0.0158	-0.014	4,769	14.9
ST-PT-LSTM	0.0158	-0.012	4,897	20.3
ST-TF-GRU	0.0158	-0.015	3,681	17.0
ST-PT-GRU	0.0158	-0.013	3,681	29.7

Key takeaways

1. **The three implementations compute the same function.** With identical weights the forward passes differ by less than $6 \cdot 10^{-7}$ and the parameter counts match exactly. After training, the core RNN on EC ends at RMSE 62.51 / 63.01 / 62.84 (scratch / Keras / PyTorch), a 0.8 % spread caused by batch order and the epoch at which early stopping fires, not by different mathematics. On ST the three core RNNs are indistinguishable (RMSE 0.0159, 0.0159, 0.0158).
2. **On EC the RNN learns real structure.** All seven recurrent models reach RMSE 62.0–63.0 against 67.9 for the train mean, 81.1 for a seasonal-naive forecast and 82.7 for persistence ($R^2 \approx 0.44$).
3. **On ST nothing beats the trivial forecast.** RMSE is 0.0158–0.0159 versus 0.0157 for predicting zero return; R^2 is slightly negative and directional accuracy (48–51 %) is below the 53.1 % of always predicting “up”. This is the expected outcome for daily returns, which are close to a random walk.
4. **Gates help little here and cost a lot.** LSTM/GRU improve EC by 0.5–1.5 % and do not help on ST, while costing 2–3.5× the time per epoch and 3–4× the parameters.
5. **Cost.** For the vanilla RNN on CPU, the NumPy loop (3.9–4.2 s/epoch) is faster than Keras

(6.2–6.5 s) and PyTorch (11.2–11.8 s) because the model is tiny; PyTorch is the fastest at inference (about 1.6 ms per 1,000 windows).

2 Theoretical Background of Recurrent Networks

This chapter develops, in the order in which the ideas depend on each other, everything needed to read the rest of the report: the forecasting task (2.1), the mathematical operators (2.2), the recurrent cell (2.3), the loss and the way a network is trained (2.4), why plain recurrence struggles (2.5), the gated cells LSTM and GRU (2.6), how a forecast is judged (2.7) and how the same layer appears in the three implementations (2.8). Every formula was also checked numerically against NumPy, finite differences, PyTorch autograd and PyTorch’s own cells; the numbers quoted are the results of those checks. Notation: N samples, T window length, F features per step, H hidden size; $x_t \in \mathbb{R}^F$ input at step t , $h_t \in \mathbb{R}^H$ hidden state, $\hat{y}$ prediction, y target, L loss. Vectors are row vectors, so a layer is written xW .

2.1 Time-Series Data and the Forecasting Problem

Sequences. In a time series the order carries information: the same value means something different depending on what preceded it. A feed-forward network receives one fixed-length vector and has no notion of order; a recurrent network reads the observations one step at a time and keeps a memory of what it has read.

Windows. A series of length L with F features is turned into a supervised data set by sliding a window of T steps: each window $(s_i, \dots, s_{i+T-1})$ is an input and the value that follows, s_{i+T} , is its target. This gives $N = L - T$ windows per series and the tensors

$$X \in \mathbb{R}^{N \times T \times F}, \quad y \in \mathbb{R}^N. \quad (1)$$

For the series $[10, 12, 11, 15, 14, 18]$ and $T = 3$ the windows are $[10, 12, 11] \rightarrow 15$, $[12, 11, 15] \rightarrow 14$ and $[11, 15, 14] \rightarrow 18$ ($N = 3$). When several series share one model (many products, many stocks) their windows are simply stacked along N .

Many-to-one and many-to-many. A many-to-one model reads T steps and produces one output from the last state; it is used throughout this report (next-step regression). A many-to-many model produces an output at every step (for example tagging each word), with the same recurrence and an output layer applied to every h_t instead of h_T only.

Chronological split. Because the future must not influence the past, the data are divided in time order into train, validation and test periods, never at random. A window may look back into an earlier period, but its target stays inside its own period. Any scaling statistics are computed on the train period only.

2.2 Mathematical Operators

- **Matrix product** $(xW)_j = \sum_i x_i W_{ij}$ mixes the inputs into new values. Example: $(1, 2) \begin{pmatrix} 1 & 0 & 2 \\ 3 & 1 & 0 \end{pmatrix} = (7, 2, 2)$.
- **Element-wise (Hadamard) product** $(a \odot b)_i = a_i b_i$ implements gates: $(0.9, 0.1) \odot (4, 4) = (3.6, 0.4)$, a gate value of 0.9 keeps 90 % of a quantity.
- **Hyperbolic tangent** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1, 1)$, with derivative $\tanh'(z) = 1 - \tanh^2(z)$. Examples: $\tanh(0) = 0$, $\tanh(1) = 0.7616$, $\tanh(3) = 0.9951$ and $\tanh'(3) = 0.0099$: a saturated unit passes almost no gradient.

- **Sigmoid** $\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1)$, with $\sigma' = \sigma(1 - \sigma) \leq 0.25$; its outputs are used as gate values.
- **Softmax** $\frac{e^{z_k}}{\sum_j e^{z_j}}$ turns scores into probabilities ($\text{softmax}(1, 2, 3) = (0.090, 0.245, 0.665)$); it is needed for classification and not for the regression heads used here.
- **Mean squared error** $L = \frac{1}{N} \sum_n (\hat{y}_n - y_n)^2$ with $\partial L / \partial \hat{y}_n = \frac{2}{N} (\hat{y}_n - y_n)$. Example for one sample: $\hat{y} = 0.8$, $y = 0.5$ gives $(0.3)^2 = 0.09$ and gradient $2 \cdot 0.3 = 0.6$.

The two derivatives above were confirmed against finite differences on a grid of points.

2.3 The Recurrent Cell

The network keeps a state $h_t \in \mathbb{R}^H$ that summarises everything read so far. At every step it combines the new input with the previous state:

$$h_t = \tanh(x_t W_{xh} + h_{t-1} W_{hh} + b), \quad h_0 = 0, \quad \hat{y} = h_T W_{hy} + b_y. \quad (2)$$

The matrices have shapes $W_{xh} \in \mathbb{R}^{F \times H}$, $W_{hh} \in \mathbb{R}^{H \times H}$, $b \in \mathbb{R}^H$, $W_{hy} \in \mathbb{R}^{H \times 1}$, $b_y \in \mathbb{R}$. *Unrolling* draws the loop as T copies of the same cell chained through the state (Figure 1). The copies share their weights, so the number of parameters

$$FH + H^2 + H \text{ (cell)} + H + 1 \text{ (head)} \quad (3)$$

does not depend on T : a window of 28 steps and a window of 1000 steps need the same 18 parameters for $F = 2$, $H = 3$ (cell only), whereas a feed-forward layer on the flattened window would grow with T .

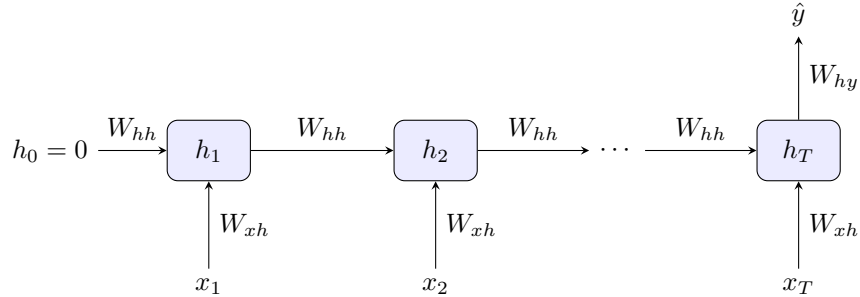


Figure 1: A many-to-one RNN unrolled over T steps. The same W_{xh} , W_{hh} and b are used at every step; only the last state feeds the output.

Worked step. Take $F = 2$, $H = 3$, $h_0 = 0$, $b = 0$, $x_1 = (1, -1)$ and $W_{xh} = \begin{pmatrix} 0.5 & -0.5 & 0.1 \\ 0.2 & 0.3 & -0.4 \end{pmatrix}$. Since $h_0 = 0$, only the input part contributes: $z_1 = x_1 W_{xh} = (0.5 - 0.2, -0.5 - 0.3, 0.1 + 0.4) = (0.3, -0.8, 0.5)$ and $h_1 = \tanh z_1 = (0.2913, -0.6640, 0.4621)$. A hand-written unrolled forward pass reproduces PyTorch's `nn.RNN` to $3 \cdot 10^{-8}$.

2.4 Training a Recurrent Network

Back-propagation through time (BPTT). Training minimises L by gradient descent. Because h_t depends on h_{t-1} , the gradient must travel backwards through every step. With $z_t = x_t W_{xh} + h_{t-1} W_{hh} + b$:

$$\frac{\partial L}{\partial h_T} = \frac{2}{N} (\hat{y} - y) W_{hy}^\top, \quad \frac{\partial L}{\partial z_t} = \frac{\partial L}{\partial h_t} \odot (1 - h_t^2), \quad \frac{\partial L}{\partial h_{t-1}} = \frac{\partial L}{\partial z_t} W_{hh}^\top, \quad (4)$$

and because the weights are shared, each of them accumulates a contribution from every step:

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^T x_t^\top \frac{\partial L}{\partial z_t}, \quad \frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^T h_{t-1}^\top \frac{\partial L}{\partial z_t}, \quad \frac{\partial L}{\partial b} = \sum_{t=1}^T \frac{\partial L}{\partial z_t}, \quad (5)$$

while $\partial L / \partial W_{hy} = h_T^\top \frac{2}{N} (\hat{y} - y)$ and $\partial L / \partial b_y = \frac{2}{N} \sum (\hat{y} - y)$. For $T = 3$ these formulas agree with central finite differences (maximum absolute error $2.9 \cdot 10^{-10}$) and with PyTorch autograd (error $\leq 2 \cdot 10^{-16}$).

Adam. The gradient g is used by the optimizer Adam with moving averages of the gradient and of its square,

$$m \leftarrow \beta_1 m + (1 - \beta_1)g, \quad v \leftarrow \beta_2 v + (1 - \beta_2)g^2, \quad \theta \leftarrow \theta - \eta \frac{m/(1 - \beta_1^k)}{\sqrt{v/(1 - \beta_2^k) + \epsilon}}, \quad (6)$$

with $\beta_1 = 0.9$, $\beta_2 = 0.999$, learning rate η and step number k . Example: for a first gradient $g = 0.5$, $m = 0.05$, $v = 0.00025$, the bias-corrected values are 0.5 and 0.25 and the step is $\eta \cdot 0.5/0.5 = \eta$: the first step has size about η whatever the scale of the gradient.

Gradient clipping. To avoid explosions the gradient is rescaled by its global norm, $g \leftarrow g \cdot \min(1, \tau/\|g\|)$. Example: two gradient blocks with norms 30 and 40 have global norm 50; with $\tau = 1$ the scale is 0.02, the clipped norm is exactly 1 and the direction is unchanged.

Early stopping. After every epoch the loss on the validation period is evaluated. Training stops when it has not improved for a fixed number of epochs (the patience) and the weights of the best epoch are restored, which protects against fitting noise.

2.5 The Vanishing and Exploding Gradient Problem

Repeating the last relation of Eq. (4) shows that the learning signal reaching step t from the output is a product of $T - t$ factors:

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^T \text{diag}(1 - h_k^2) W_{hh}^\top, \quad \left\| \frac{\partial L}{\partial h_t} \right\| \lesssim (\rho c)^{T-t} \left\| \frac{\partial L}{\partial h_T} \right\|, \quad (7)$$

where ρ is the spectral radius of W_{hh} and $c \leq 1$ is the typical value of $1 - h^2$. If $\rho c < 1$ the signal *vanishes* exponentially and early inputs cannot influence learning (a factor 0.5 per step leaves $0.5^{20} \approx 10^{-6}$ after 20 steps); if $\rho c > 1$ it *explodes* ($1.5^{20} \approx 3300$) and the updates become destructive. A numeric experiment with $T = 100$ confirms both regimes: with spectral radius 0.5 the gradient at the first step is below 10^{-10} of the last-step gradient, with radius 3 it is above 10^3 . Clipping (Section 2.4) controls explosions; the architecture must be changed to counter vanishing, which is what the gated cells do.

2.6 Gated Cells: LSTM and GRU

A *gate* is a sigmoid output multiplied element-wise onto a quantity, so the network decides how much of it to keep, write or expose.

LSTM. It carries two states, the hidden state h_t and the cell state c_t (the long-term memory):

$$\begin{aligned} i_t &= \sigma(x_t W_i + h_{t-1} U_i + b_i), & f_t &= \sigma(x_t W_f + h_{t-1} U_f + b_f), \\ g_t &= \tanh(x_t W_g + h_{t-1} U_g + b_g), & o_t &= \sigma(x_t W_o + h_{t-1} U_o + b_o), \\ c_t &= f_t \odot c_{t-1} + i_t \odot g_t, & h_t &= o_t \odot \tanh(c_t), \end{aligned} \quad (8)$$

with input gate i_t (how much new content to write), forget gate f_t (how much memory to keep), candidate g_t and output gate o_t . The update of c_t is *additive*: $\partial c_t / \partial c_{t-1} = f_t$, so with $f_t \approx 1$

the gradient passes unchanged through many steps, which counters the vanishing of Section 2.5. Example: $f = 0.9$, $c_{t-1} = 2$, $i = 0.5$, $g = 0.4$ gives $c_t = 0.9 \cdot 2 + 0.5 \cdot 0.4 = 2.0$.

GRU. It merges the two states: with reset gate r_t , update gate z_t and candidate $n_t = \tanh(x_t W_n + b_{in} + r_t \odot (h_{t-1} U_n + b_{hn}))$,

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}, \quad (9)$$

a convex mixture of the new candidate and the old state, so memory can be preserved when $z_t \approx 1$.

Cost. A vanilla cell has $FH + H^2 + H$ parameters; a GRU has three such blocks and an LSTM four (one per gate and candidate), so with $F = 10$, $H = 32$ the recurrent parameters are 1,376, 4,128 and 5,504. Step-by-step hand computations of an LSTM and a GRU step match PyTorch’s `LSTMCell` and `GRUCell` to within $6 \cdot 10^{-8}$.

2.7 Evaluating a Forecast: Baselines and Metrics

A recurrent network is only useful if it beats simple forecasts, so every model is compared with baselines: *persistence* ($\hat{y}_t = y_{t-1}$), *seasonal naïve* ($\hat{y}_t = y_{t-m}$ for a period m), the *train mean* and, for returns, *zero return*. The metrics, all computed on the test period, are

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum (y - \hat{y})^2}, \quad \text{MAE} = \frac{1}{N} \sum |y - \hat{y}|, \quad R_{\text{os}}^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y}_{\text{train}})^2}, \quad (10)$$

where R_{os}^2 compares the model with the predictor that always outputs the train mean ($R_{\text{os}}^2 < 0$ means worse than that predictor). For returns two further scores are used: the *directional accuracy*, the share of windows where $\text{sign } \hat{y} = \text{sign } y$ (to be read against the share of the most frequent sign), and the *information coefficient* (IC), the correlation between $\hat{y}$ and y . RMSE and MAE are reported in the original units (after undoing any scaling or logarithm), R_{os}^2 in the scaled space in which the models are trained.

2.8 The Same Layer in Three Implementations

The same equations can be written by hand (NumPy), with a high-level API (Keras) or with a flexible framework (PyTorch). They differ in how much of the loop the user writes, in the layout of the weights and in the number of biases (Table 2); with the same weights they compute the same function.

Table 2: The same recurrent layer in the three implementations.

	Scratch	Keras SimpleRNN(H)	PyTorch nn.RNN
Input	(N, T, F)	(N, T, F)	(N, T, F) with <code>batch_first=True</code>
W_{xh}	(F, H)	<code>kernel</code> (F, H)	<code>weight_ih_10</code> (H, F) , transposed
W_{hh}	(H, H)	<code>recurrent_kernel</code>	<code>weight_hh_10</code> , transposed
Bias	one vector	one vector	two vectors (<code>bias_ih</code> , <code>bias_hh</code>)
Output	last state	last state	all states, <code>take out[:, -1]</code>
Default init	Glorot / orthogonal	Glorot / orthogonal	uniform $\pm 1/\sqrt{H}$
LSTM gate order	–	i, f, c, o	i, f, g, o
GRU gate order	–	z, r, h	r, z, n
Training loop	hand-written BPTT + Adam	<code>fit()</code>	hand-written loop, <code>backward()</code>

3 Data: Sources, Features and Splits

Both datasets contain about 430,000 windows, and their raw files are below 100 MB.

3.1 EC: Order Flow of an Online Retailer (Online Retail II)

Source: UCI Online Retail II (Kaggle [mashlyn/online-retail-ii-uci](#)), a UK online retailer, 2009-12-01 to 2011-12-09, 1,067,371 invoice lines. Every invoice is an *order*; an invoice whose number starts with C is a *cancellation or return*. After removing 34,335 exact duplicate lines, real orders (not C, quantity > 0, price > 0) define **604 open days** (the shop never sells on Saturdays and closes over Christmas; those days are dropped, not filled with zero, and the feature **gap** records the calendar days since the previous open day). The 750 products with the most distinct invoices are kept, giving $750 \times (604 - 29) = 432,000$ windows of $T = 28$ open days and $F = 10$ features.

Table 3: EC features (per product and open day).

Feature	Meaning	Transform
units (target)	units ordered	$\log(1 + x)$, z-score
orders, customers	distinct invoices / distinct customer IDs (22 % of lines have no ID)	$\log(1 + x)$, z-score
cancel_units	units in cancellation invoices (service-quality signal)	$\log(1 + x)$, z-score
price	mean unit price	log, z-score
gap	days since previous open day	z-score
dow, doy sin/cos	weekday and day of year on a circle	none

The log transform turns ratios into differences (0, 9 and 99 units become 0, 2.30 and 4.61), so a ten-fold jump looks like a constant step. Task: the last 28 open days $\rightarrow$ units on the next open day, one model for all products. Baselines: *persistence* ($y_t = y_{t-1}$), *seasonal naive* ($y_t = y_{t-6}$, one open-day week) and the train mean.

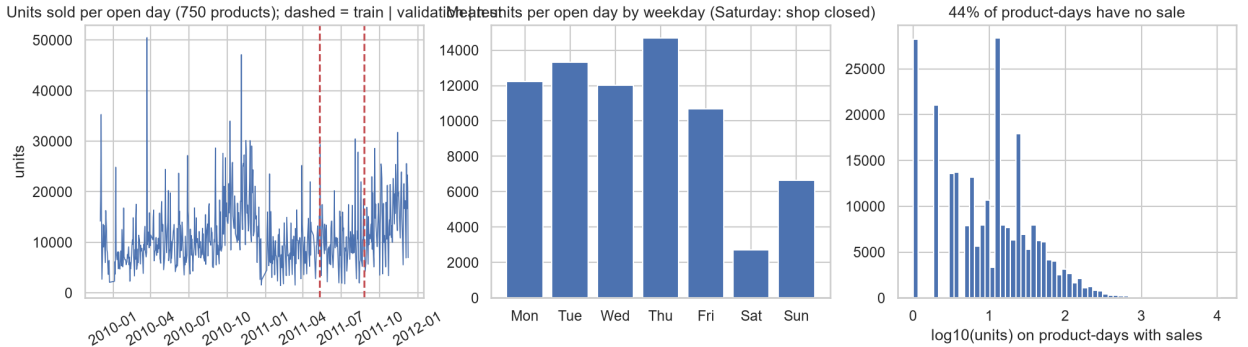


Figure 2: EC: total units per open day (dashed lines: train | validation | test), mean units by weekday (Saturday: shop closed) and the distribution of units per product-day. A large share of product-days has no sale at all.

3.2 ST: Daily Returns of S&P 500 Stocks

Source: Kaggle [camnugent/sandp500](#), `all_stocks_5yr.csv`, 619,040 rows, 505 tickers, 2013-02-08 to 2018-02-07. 470 tickers have the full 1,259-day history; the 350 with the highest mean dollar volume (close $\times$ volume) are kept, giving $350 \times (1258 - 30) = 429,800$ windows of $T = 30$ days and $F = 4$ features. Missing open/high/low values (a handful) are replaced by the day's close.

Table 4: ST features (per stock and day).

Feature	Meaning	Transform
ret (target)	$\ln(\text{close}_t / \text{close}_{t-1})$; $+1\% \rightarrow 0.00995$, $-1\% \rightarrow -0.01005$	z-score
hl_range	$(\text{high} - \text{low}) / \text{close}$, intraday volatility	z-score
oc_ret	$\ln(\text{close} / \text{open})$	z-score
vol_z	$\ln(1 + \text{volume})$	z-score per ticker

Task: the last 30 days $\rightarrow$ log return on the next day, one pooled model for all stocks. Baselines: zero return, previous-day return, always “up” and the train mean. The EDA (Figure 3) shows why the task is hard: returns have excess kurtosis 54.2, lag-1 autocorrelation -0.005 , while *squared* returns have lag-1 autocorrelation 0.094: volatility clusters, direction does not.

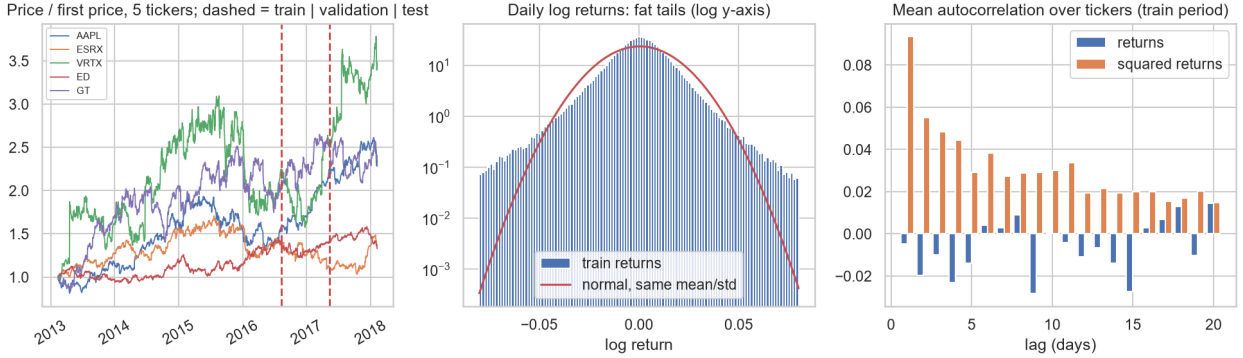


Figure 3: ST: normalised prices of five stocks with the split dates, heavy-tailed return distribution against a normal curve (log axis), and mean autocorrelation of returns and squared returns.

3.3 Splitting, Scaling and Evaluation Protocol

- **Chronological split by target day:** first 70 % of days train, next 15 % validation, last 15 % test (EC: $\text{train} < 2011-05-10 \leq \text{validation} < 2011-08-25 \leq \text{test}$; ST: $\text{train} < 2016-08-09 \leq \text{validation} < 2017-05-10 \leq \text{test}$). Window counts: EC 295,500 / 68,250 / 68,250, ST 297,500 / 66,150 / 66,150. A window may look back into an earlier split, but its target never leaves its own split.
- **Scaling** (mean and standard deviation) is computed on the train period only.
- **Metrics:** as defined in Section 2.7. EC: RMSE and MAE in units sold (after undoing the transforms), R^2_{oos} on the log scale. ST: RMSE and MAE in log-return, R^2_{oos} , directional accuracy and information coefficient.

4 Model Architecture and Parameters

4.1 Architecture

The core model, identical in all three implementations, is: input $(N, T, F) \rightarrow$ one vanilla RNN layer with $H = 32$ units and tanh activation $\rightarrow$ last hidden state $h_T \rightarrow$ one linear output unit. It is trained with the mean squared error on the scaled target. LSTM and GRU variants (also $H = 32$, one layer) are trained in Keras and PyTorch with the frameworks’ default initialisation.

4.2 Hyperparameters and Configuration

Table 5: All hyperparameters and configuration values. They are the same for every run unless two values are given (EC / ST).

Name	Value	Meaning
Series used	750 / 350	products (EC) / stocks (ST) pooled into one model
Time steps	604 / 1,258	open days (EC) / trading days with a return (ST)
Window length T	28 / 30	steps read before each prediction
Features per step F	10 / 4	see Section 3
Windows N (train / val. / test)	295,500 / 68,250 / 68,250 (EC); 297,500 / 66,150 / 66,150 (ST)	chronological 70 / 15 / 15 split by target day
Recurrent layers	1	one cell applied over the T steps
Hidden size H	32	dimension of h_t
Activation	tanh	inside the cell; the output unit is linear
Dropout, weight decay	none	no regularisation other than early stopping
Loss	MSE	on the z-scored target
Optimizer	Adam	$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-7}$
Learning rate η	10^{-3}	constant
Batch size	128	windows per update; shuffled within the train period each epoch
Maximum epochs	30	upper limit of passes over the train windows
Early stopping	patience 5	stop after 5 epochs without improvement of the validation loss; best weights restored
Gradient clipping τ	1.0	global norm of all gradients
Initial W_{xh}	Glorot uniform	$U(-\sqrt{6/(F+H)}, \sqrt{6/(F+H)})$
Initial W_{hh}	orthogonal	from the QR decomposition of a Gaussian matrix
Initial W_{hy}	Glorot uniform	$U(-\sqrt{6/(H+1)}, \sqrt{6/(H+1)})$
Initial biases	0	b and b_y
Random seed	42	Python, NumPy, Keras and PyTorch
Numeric precision	float32	gradient check in float64

4.3 Trainable Parameters

Table 6: Trainable parameters of the core RNN (identical in scratch, Keras and PyTorch).

Parameter	Shape	EC ($F = 10$)	ST ($F = 4$)
Input weights W_{xh}	$F \times H$	320	128
Recurrent weights W_{hh}	$H \times H$	1,024	1,024
Cell bias b	H	32	32
Output weights W_{hy}	$H \times 1$	32	32
Output bias b_y	1	1	1
Total	$FH + H^2 + H + H + 1$	1,409	1,217

The gated variants have more parameters because each gate and candidate has its own W , U and bias block (Section 2.6):

Table 7: Trainable parameters of all model types (one recurrent layer with $H = 32$ plus the output unit).

Model	EC		ST	
	Keras	PyTorch	Keras	PyTorch
RNN	1,409	1,409	1,217	1,217
GRU	4,257	4,257	3,681	3,681
LSTM	5,537	5,665	4,769	4,897

PyTorch’s `nn.LSTM` keeps two bias vectors per gate block, i.e. $4H = 128$ more than Keras’s LSTM; the GRU counts coincide because Keras’s GRU also uses two biases (`reset_after=True`). For the core RNN the second PyTorch bias is zeroed and frozen so that the three counts are equal.

4.4 Initialisation and Naming

Same weights. The scratch model draws Glorot-uniform W_{xh} , orthogonal W_{hh} and zero biases. The same arrays are copied into Keras (`set_weights`) and PyTorch (transposed). PyTorch has a second bias, which is zeroed and frozen so that the parameter counts are equal; this is asserted in code. Because the frameworks shuffle with their own generators and Adam’s ϵ is placed slightly differently in each, training is not bit-identical across implementations; only the starting point is.

Naming. Run ID = `<dataset>-<implementation>-<model>` with dataset EC/ST, implementation SC (scratch), TF (Keras), PT (PyTorch) and model RNN/LSTM/GRU. The same ID names the result file, the figures and the table rows.

5 Implementation Details

5.1 NumPy Implementation from Scratch

The class `ScratchRNN` stores the five arrays $W_{xh}, W_{hh}, b, W_{hy}, b_y$. The forward pass precomputes XW_{xh} for all steps at once and loops only over time; the backward pass loops from $t = T$ to 1 applying Eq. (4), accumulating the shared-weight gradients, and also returns $\|\partial L / \partial h_t\|$ for diagnostics. Adam and global-norm clipping are written by hand.

Gradient check. Before training, every parameter of the scratch model is checked against central finite differences ($\epsilon = 10^{-6}$, float64, 8 real windows). The error is measured per parameter as $\max |a - n| / \max |a|$:

Table 8: Gradient check: BPTT versus finite differences (all entries below the 10^{-5} threshold).

Parameter	EC (error)	ST (error)
W_{xh}	$3.8 \cdot 10^{-10}$	$8.6 \cdot 10^{-10}$
W_{hh}	$9.3 \cdot 10^{-10}$	$8.3 \cdot 10^{-10}$
b	$4.4 \cdot 10^{-10}$	$1.6 \cdot 10^{-9}$
W_{hy}	$2.1 \cdot 10^{-10}$	$3.4 \cdot 10^{-10}$
b_y	$5.0 \cdot 10^{-12}$	$1.9 \cdot 10^{-10}$

5.2 Keras Implementation

`Sequential([Input((T,F)), SimpleRNN(32), Dense(1)])`, compiled with Adam (`global_clipnorm=1.0`, $\epsilon = 10^{-7}$) and MSE; `fit` runs shuffled mini-batches with `EarlyStopping(restore_best_weights=True)`. LSTM and GRU replace the first layer.

5.3 PyTorch Implementation

A module with `nn.RNN(F, 32, batch_first=True)` and `nn.Linear(32, 1)` applied to `out[:, -1]`; a hand-written loop does `zero_grad` $\rightarrow$ `forward` $\rightarrow$ `backward` $\rightarrow$ `clip_grad_norm` $\rightarrow$ `step` and restores the best state. LSTM and GRU use `nn.LSTM/nn.GRU`. These keep two biases, hence their counts differ slightly from Keras’s LSTM (by $4H = 128$); GRU matches because Keras uses `reset_after=True`.

5.4 Verification That the Three Implementations Agree

With the same weights, the three forward passes on 64 validation windows differ by at most $4.8 \cdot 10^{-7}$ (EC: scratch–Keras $4.8 \cdot 10^{-7}$, scratch–PyTorch $4.8 \cdot 10^{-7}$, Keras–PyTorch $6.0 \cdot 10^{-7}$; ST: 4.8, 3.6, $3.3 \cdot 10^{-7}$), which is float32 round-off, and the parameter counts are identical (1,409 and 1,217).

6 Experiment 1: Forecasting Order Volume (EC)

Table 9: EC, test period (68,250 windows). RMSE and MAE in units sold; R^2 on the log scale; epochs run (best epoch); s/epoch = median over epochs 2+; ms/1k = inference time per 1,000 windows.

Run	RMSE	MAE	R^2	params	epochs (best)	s/epoch	ms/1k
<i>persistence</i> ($y[t]=y[t-1]$)	82.68	23.91	-0.044	–	–	–	–
<i>seasonal naive</i> ($y[t]=y[t-6]$)	81.06	23.19	-0.030	–	–	–	–
<i>train mean</i>	67.88	19.26	0.000	–	–	–	–
EC-SC-RNN	62.51	15.90	0.439	1,409	12 (7)	3.9	5.54
EC-TF-RNN	63.01	16.02	0.438	1,409	22 (17)	6.2	8.02
EC-PT-RNN	62.84	16.00	0.439	1,409	14 (9)	11.2	1.54
EC-TF-LSTM	62.44	15.93	0.446	5,537	12 (7)	14.6	16.90
EC-PT-LSTM	62.03	15.83	0.447	5,665	18 (13)	22.0	4.14
EC-TF-GRU	62.41	15.89	0.449	4,257	16 (11)	16.4	15.05
EC-PT-GRU	62.24	15.87	0.448	4,257	15 (10)	28.4	5.12

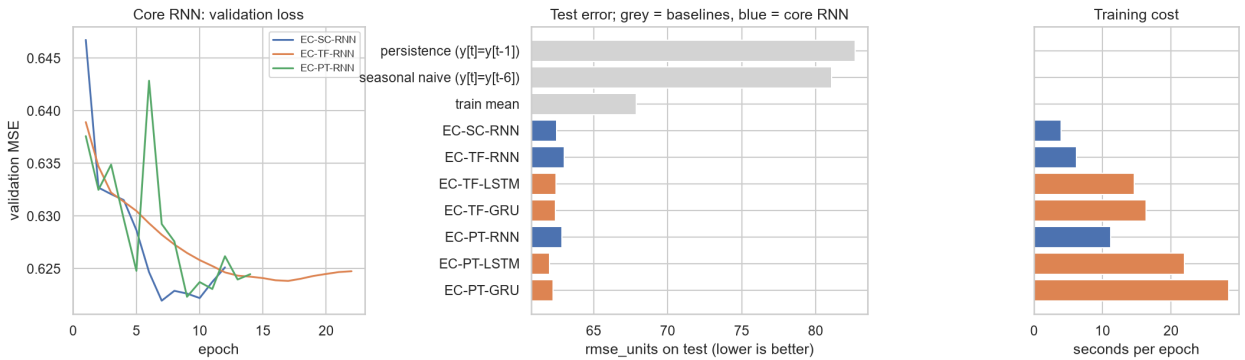


Figure 4: EC: validation loss of the three core RNNs, test RMSE of all runs against the baselines (grey), and time per epoch.

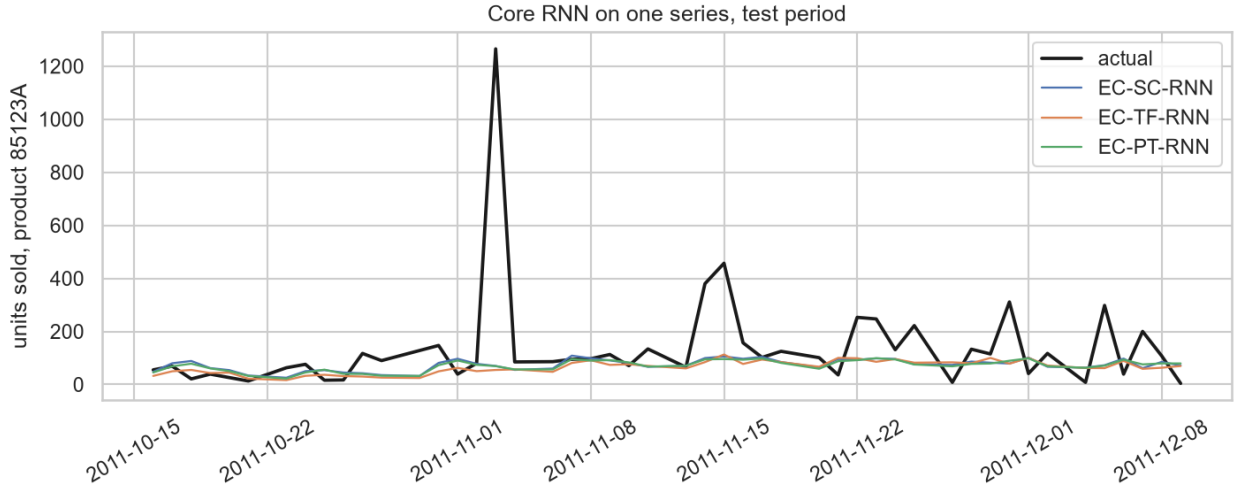


Figure 5: EC: forecasts of the three core RNNs for the most-ordered product over the last 48 test days. The models follow the level and the weekly rhythm but, as with any regression to the mean, not the one-off spike.

Discussion.

- **RNNs beat every baseline.** Persistence (82.7) and seasonal naive (81.1) are worse than the train mean (67.9) because demand is intermittent and spiky; the RNNs (62.0–63.0) are 8–9 % below the mean and 24–25 % below persistence in RMSE.
- **Implementations agree.** Core RNN RMSE: scratch 62.51, Keras 63.01, PyTorch 62.84, with R^2 0.439 / 0.438 / 0.439. The validation curves (left panel of the comparison figure) have different shapes because of different batch order; Keras stops at epoch 22 (best 17), scratch at 12 (best 7), PyTorch at 14 (best 9).
- **Gates:** LSTM/GRU give 62.0–62.4 (best EC-PT-LSTM, 62.03) against 62.5–63.0 for the vanilla RNN, i.e. 0.5–1.5 % better, for 3–4× the parameters and 2–3.5× the time per epoch.
- **Memory.** The gradient reaching the first of the 28 days is 0.1 % of the last-step gradient at initialisation and 13.7 % after training: training lengthened the effective memory.
- **Cost.** Vanilla RNN, seconds per epoch: scratch 3.9, Keras 6.2, PyTorch 11.2; inference per 1,000 windows: 5.5, 8.0 and 1.5 ms.

7 Experiment 2: Forecasting Stock Returns (ST)

Table 10: ST, test period (66,150 windows). RMSE in log-return; R^2 against the train-mean predictor; dir. acc. = directional accuracy; IC = information coefficient. The “always up” rate on this test period is 0.531 (the train mean is positive, so it predicts “up” everywhere; the zero predictors have no sign).

Run	RMSE	R^2	dir. acc.	IC	params	epochs (best)	s/epoch	ms/1k
<i>zero return</i>	0.0157	-0.001	—	—	—	—	—	—
<i>always up (+1e-6)</i>	0.0157	-0.001	—	—	—	—	—	—
<i>previous-day return</i>	0.0223	-1.016	0.492	-0.011	—	—	—	—
<i>train mean</i>	0.0157	0.000	0.531	—	—	—	—	—
ST-SC-RNN	0.0159	-0.017	0.508	-0.015	1,217	7 (2)	4.2	6.19
ST-TF-RNN	0.0159	-0.017	0.483	-0.021	1,217	6 (1)	6.5	7.36
ST-PT-RNN	0.0158	-0.016	0.496	-0.027	1,217	6 (1)	11.8	1.63
ST-TF-LSTM	0.0158	-0.014	0.483	-0.021	4,769	6 (1)	14.9	18.19
ST-PT-LSTM	0.0158	-0.012	0.493	-0.022	4,897	6 (1)	20.3	3.95
ST-TF-GRU	0.0158	-0.015	0.483	-0.023	3,681	6 (1)	17.0	16.60
ST-PT-GRU	0.0158	-0.013	0.485	-0.028	3,681	6 (1)	29.7	5.52

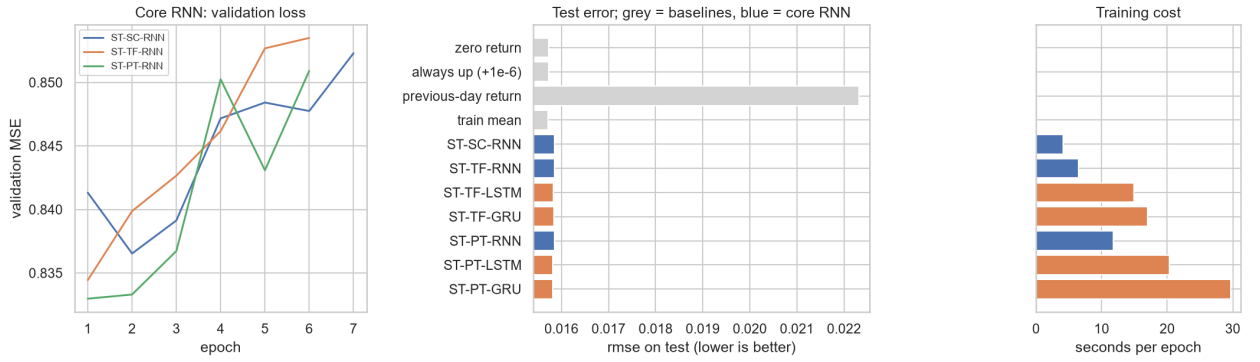


Figure 6: ST: validation loss of the core RNNs, test RMSE of all runs against the baselines (grey), and time per epoch. The bar scale is truncated, so the differences look larger than they are (all seven runs are within 0.3% of each other).

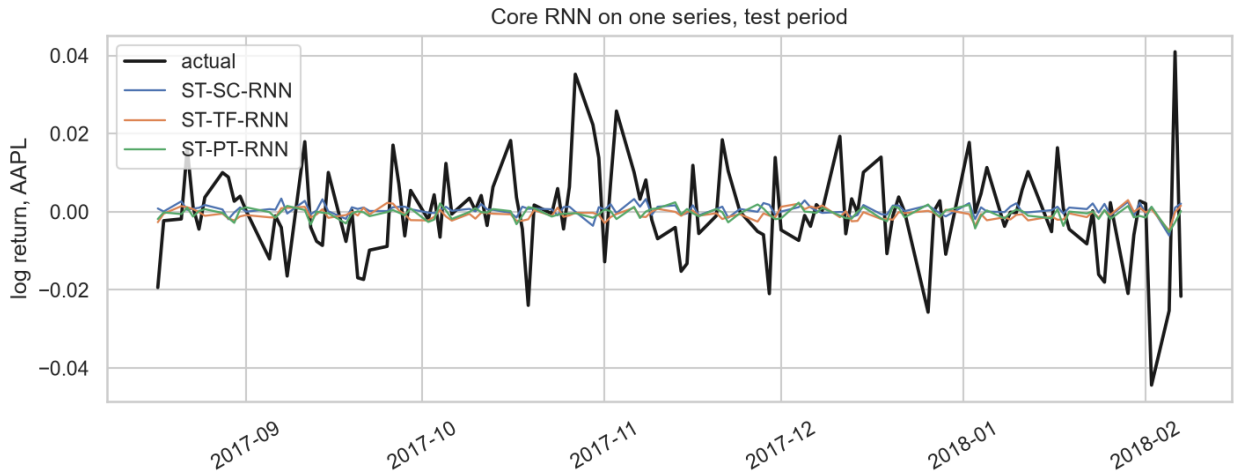


Figure 7: ST: predicted and actual daily log return for AAPL over the last 120 test days. The predictions hug zero: the models learn that they cannot predict.

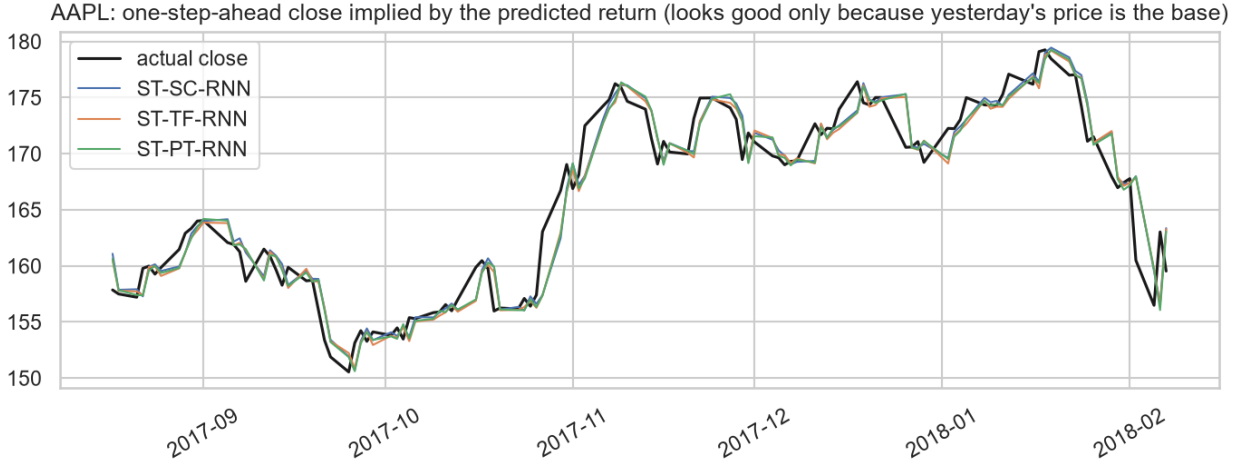


Figure 8: ST: one-step-ahead close implied by the predicted return for AAPL. All models track the price closely only because the previous close is the base of the forecast.

Discussion.

- **No model beats the trivial forecast.** RMSE 0.0158–0.0159 against 0.0157 for zero return; R^2 -0.012 to -0.017 ; IC -0.015 to -0.028 . Predicting yesterday's return is far worse (RMSE 0.0223, $R^2 = -1.02$).
- **Direction is not predictable.** Directional accuracy is 48.3–50.8%, below the 53.1% of always saying “up” in a rising test period. Every run reaches its best validation loss at epoch 1–2 and then only fits noise.
- **Why.** Returns are almost uncorrelated with their past (lag-1 autocorrelation -0.005) and heavy-tailed (excess kurtosis 54), so MSE is dominated by a few extreme days; only the *size* of moves (volatility) is predictable and that is not what an MSE-trained point forecast of the *sign* uses.
- **Implementations still agree:** RMSE 0.0159 / 0.0159 / 0.0158, R^2 -0.017 / -0.017 / -0.016 . LSTM/GRU do not help (R^2 -0.012 to -0.015) and cost $2\text{--}3.5\times$ more per epoch.
- **Price view.** Turning predicted returns into prices ($\hat{P}_{t+1} = P_t e^{\hat{r}}$) makes every model look excellent because yesterday's price carries almost all the information (Figure 8); the return-space metrics are the meaningful ones.

8 Comparison of Implementations and Datasets

- **Same code, opposite outcomes.** The architecture, settings and implementations are identical on EC and ST, yet EC is learnable ($R^2 \approx 0.44$) and ST is not ($R^2 < 0$). The difference lies in the data: weekly and yearly rhythm, order and cancellation counts carry information about next-day demand; daily stock returns carry almost none about the next return. The stock result is therefore a negative one, measured against the right baselines.
- **Scratch = Keras = PyTorch in function.** On both datasets the three core RNNs start from the same function and end within 0.8% (EC) and 0.03% (ST) of each other; the abstraction level changes the amount of code and the speed, not the learned model.
- **Gates** improve a series with long, regular patterns a little (EC) and do nothing for noise (ST).
- **Speed.** On CPU with $H = 32$ the NumPy loop is the fastest to train; PyTorch pays Python

overhead per mini-batch in its hand-written loop (and per-op overhead on tiny tensors), Keras sits in between; PyTorch is the fastest for inference.

9 Conclusions and Limitations

This work covered the concepts of an RNN (sequence tensors, operators, recurrence, BPTT, vanishing gradients and clipping, LSTM/GRU gates), two time-related datasets of about 430,000 windows each, and three implementations of the same RNN (scratch, Keras, PyTorch) that were verified to be the same function (forward-pass agreement $< 10^{-6}$, gradient check $< 10^{-9}$, equal parameter counts) and compared fairly. On e-commerce order flow the RNN beats all baselines; on stock returns it correctly learns that there is nothing to predict.

Limitations. One seed and one chronological split per dataset: differences below about 1% between runs should not be read as a ranking. Hidden size and window length were fixed, not tuned or varied. The ST pool contains survivors of the index (stocks with complete histories), and results describe 2013–2018 only. Timings are CPU-only (TensorFlow has no GPU build on native Windows), so the speed ranking may change on a GPU. The scratch implementation covers the vanilla RNN only; LSTM and GRU exist in Keras and PyTorch.

Code and results. The three notebooks (`00_rnn_concepts`, `ec_retail`, `st_sp500`) run top to bottom, each dataset notebook in about 30 minutes on a CPU. Metrics and training histories of every run are stored as JSON files, and the tables of this report are generated from them. Software: Python 3.13, TensorFlow 2.21 / Keras 3.15, PyTorch 2.14, NumPy 2.5.